gRPC in klyntar-server: A Complete Guide

This document explains every gRPC-related change made during the migration from REST to gRPC in this project. It's written as a learning reference — if you understand everything here, you'll be able to add new gRPC services to klyntar-server on your own.

Table of Contents

---

0. Setting Up gRPC in a Go Project From Scratch

This section walks you through every step of adding gRPC to a Go project, starting from nothing. If you're starting a new project or adding gRPC to an existing one, follow these steps in order.

0.1 Prerequisites

You need **Go 1.21+** installed. Verify:

```
go version
# go version go1.25.0 ...
```

You also need a Go module initialized. If you're starting fresh:

```
mkdir my-grpc-project && cd my-grpc-project
go mod init example.com/my-grpc-project
```

If you already have a Go project with `go.mod`, skip this.

0.2 Install Go Dependencies

You need two runtime libraries:

```
# The gRPC framework — provides the server, client, and transport layer
go get google.golang.org/grpc

# The protobuf runtime — provides the generated message types
go get google.golang.org/protobuf
```

What each one does:

Package	Purpose	Used where
`google.golang.org/grpc`	gRPC server (`grpc.NewServer()`), client (`grpc.Dial()`), status codes, interceptors	`main.go`, handler files, client code
`google.golang.org/protobuf`	Runtime support for generated protobuf structs (marshal/unmarshal, reflection)	Imported automatically by generated `*.pb.go` files

After running these, your `go.mod` will include:

```
require (
    google.golang.org/grpc v1.79.3
    google.golang.org/protobuf v1.36.11
)
```

Several indirect dependencies will also appear (like `golang.org/x/net` for HTTP/2 support) — these are pulled in automatically, you don't need to manage them.

0.3 Install buf (Protobuf Toolchain)

`buf` is the tool that reads your `.proto` files and generates Go code. Install it for your platform:

```
# Windows (winget)
winget install bufbuild.buf

# macOS (Homebrew)
brew install bufbuild/buf/buf

# Linux (direct download)
# See https://buf.build/docs/installation

# Verify installation
buf --version
# 1.66.1
```

Why buf instead of protoc?

The older approach uses `protoc` (the protobuf compiler) with manually installed plugins (`protoc-gen-go`, `protoc-gen-go-grpc`). This requires:

- Installing `protoc` separately
- Installing each plugin via `go install`
- Wiring them together with complex command-line flags

`buf` replaces all of that. It downloads plugins automatically (via `remote:` in `buf.gen.yaml`), manages versions, and adds linting and breaking change detection. For reference, here's what the old `protoc` approach looks like — you don't need to do this if you use buf:

```
# OLD WAY (don't do this if you have buf):
# Install protoc compiler from
https://github.com/protocolbuffers/protobuf/releases
# Install the Go plugins:
go install google.golang.org/protobuf/cmd/protoc-gen-go@latest
go install google.golang.org/grpc/cmd/protoc-gen-go-grpc@latest
# Run protoc with flags:
protoc --go_out=gen/pb --go_opt=paths=source_relative \
       --go-grpc_out=gen/pb --go-grpc_opt=paths=source_relative \
       -I api/proto \
       api/proto/users/v1/users.proto
```

0.4 Create the buf Configuration Files

You need two files in your project root:

`buf.yaml` — project-level config. Tells buf where your proto files live:

```
version: v2
modules:
  - path: api/proto
lint:
```

```
    use:
      - DEFAULT
  breaking:
    use:
      - FILE
```

Field	Meaning
`modules[].path`	Directory containing your `.proto` files. All `import` paths in proto files are relative to this.
`lint.use: [DEFAULT]`	Apply standard protobuf style rules (e.g., field names must be snake_case, service names must be PascalCase).
`breaking.use: [FILE]`	When running `buf breaking`, detect changes that would break existing clients (renamed fields, changed types, etc.).

`buf.gen.yaml` — code generation config. Tells buf what to generate and where:

```
version: v2
plugins:
  - remote: buf.build/protocolbuffers/go
    out: gen/pb
    opt:
      - paths=source_relative
  - remote: buf.build/grpc/go
    out: gen/pb
    opt:
      - paths=source_relative
```

Field	Meaning
`plugins[].remote`	The code generation plugin, downloaded from the Buf Schema Registry. No local install needed.
`plugins[].out`	Output directory for generated files.
`plugins[].opt`	Plugin options. `paths=source_relative` means the output directory structure mirrors the proto file directory structure.

The two plugins and what they generate:

Plugin	Generated file	Contents
`buf.build/protocolbuffers/go`	`*.pb.go`	Go structs for each `message`, plus marshal/unmarshal methods
`buf.build/grpc/go`	`*_grpc.pb.go`	Server interface, client stub, and registration function for each `service`

Important: Make sure `gen/` is in your `.gitignore` if you want to regenerate on build, or commit it if you want the project to compile without buf installed. This project commits the generated code.

0.5 Create the Directory Structure

```
# Proto source files go here (matches buf.yaml modules[].path)
mkdir -p api/proto

# Generated code will appear here (matches buf.gen.yaml plugins[].out)
# Don't create this manually — buf generate creates it
# mkdir -p gen/pb

# Your handler implementations go here
mkdir -p internal
```

The recommended layout for proto files follows this pattern:

```
api/proto/
└── <service-name>/
    └── <version>/
        └── <service-name>.proto
```

For example:

```
api/proto/
├── users/v1/users.proto
├── voids/v1/voids.proto
├── posts/v1/posts.proto
└── messages/v1/messages.proto
```

Why the `v1` directory? API versioning. If you need to make breaking changes later, you create `v2/` side by side. Clients on v1 keep working while you migrate them.

0.6 Write Your First Proto File

Create `api/proto/greeter/v1/greeter.proto` (a minimal example):

```
syntax = "proto3";
package myproject.greeter.v1;

option go_package = "example.com/my-grpc-project/gen/pb/greeter/v1";

// The service definition — becomes a Go interface
service GreeterService {
  rpc SayHello(SayHelloRequest) returns (SayHelloResponse);
}
```

```
// Request message — becomes a Go struct
message SayHelloRequest {
  string name = 1;
}

// Response message — becomes a Go struct
message SayHelloResponse {
  string greeting = 1;
}
```

Checklist for every proto file:

- ☐ `syntax = "proto3";` on line 1
- ☐ `package` matches directory structure: `<project>.<service>.<version>`
- ☐ `option go_package` points to where generated code will land (must match `buf.gen.yaml out` + directory path)
- ☐ At least one `service` with at least one `rpc`
- ☐ Every `rpc` has a dedicated request and response message (even if empty — use `google.protobuf.Empty` for truly empty responses)
- ☐ Field numbers start at 1 and are unique within each message

0.7 Generate Go Code

```
buf generate
```

That's it. Check the output:

```
ls gen/pb/greeter/v1/
# greeter.pb.go        <- structs (SayHelloRequest, SayHelloResponse)
# greeter_grpc.pb.go   <- interface (GreeterServiceServer) + client
(NewGreeterServiceClient)
```

Common issues:

Error	Cause	Fix
`"api/proto" had no .proto files`	Empty proto directory	Make sure proto files exist in the `modules[].path` directory
`go_package option is missing`	Forgot `option go_package` in proto file	Add the `option go_package = "...";` line
`import "..." not found`	Importing another proto file that doesn't exist	Check the import path is relative to `api/proto/`

Error	Cause	Fix
`field number X is already used`	Duplicate field numbers in a message	Each field in a message must have a unique number

Run this every time you change a `.proto` file. Forgetting to regenerate is the #1 source of confusion — your Go code will be out of sync with your proto definitions.

0.8 Implement the Server

Now you write the actual Go code. There are three pieces:

Piece 1: The handler — implements the generated interface

Create `internal/greeter/handler.go`:

```
package greeter

import (
    "context"
    "fmt"

    greeterpb "example.com/my-grpc-project/gen/pb/greeter/v1"
)

type Handler struct {
    greeterpb.UnimplementedGreeterServiceServer // REQUIRED: embed this
}

func NewHandler() *Handler {
    return &Handler{}
}

func (h *Handler) SayHello(ctx context.Context, req *greeterpb.SayHelloRequest)
(*greeterpb.SayHelloResponse, error) {
    return &greeterpb.SayHelloResponse{
        Greeting: fmt.Sprintf("Hello, %s!", req.GetName()),
    }, nil
}
```

Piece 2: The main function — creates and starts the gRPC server

Create `cmd/server/main.go`:

```
package main

import (
    "fmt"
    "log"
    "net"
```

```
    greeterpb "example.com/my-grpc-project/gen/pb/greeter/v1"
    "example.com/my-grpc-project/internal/greeter"
    "google.golang.org/grpc"
    "google.golang.org/grpc/reflection"
)

func main() {
    // 1. Open a TCP port
    lis, err := net.Listen("tcp", ":50051")
    if err != nil {
        log.Fatalf("failed to listen: %v", err)
    }

    // 2. Create a gRPC server instance
    grpcServer := grpc.NewServer()

    // 3. Register your handler
    greeterpb.RegisterGreeterServiceServer(grpcServer, greeter.NewHandler())

    // 4. Enable reflection (lets grpcurl/evans discover your API)
    reflection.Register(grpcServer)

    // 5. Start serving
    fmt.Println("gRPC server listening on :50051")
    if err := grpcServer.Serve(lis); err != nil {
        log.Fatalf("failed to serve: %v", err)
    }
}
```

Piece 3 (optional): A Go client — for other Go services to call yours

```
package main

import (
    "context"
    "fmt"
    "log"

    greeterpb "example.com/my-grpc-project/gen/pb/greeter/v1"
    "google.golang.org/grpc"
    "google.golang.org/grpc/credentials/insecure"
)

func main() {
    // Connect to the server
    conn, err := grpc.NewClient("localhost:50051",
        grpc.WithTransportCredentials(insecure.NewCredentials()),
    )
    if err != nil {
        log.Fatalf("failed to connect: %v", err)
    }
```

```
    defer conn.Close()

    // Create a typed client (generated by buf)
    client := greeterpb.NewGreeterServiceClient(conn)

    // Call the RPC — just like calling a local function
    resp, err := client.SayHello(context.Background(), &greeterpb.SayHelloRequest{
        Name: "World",
    })
    if err != nil {
        log.Fatalf("SayHello failed: %v", err)
    }
    fmt.Println(resp.GetGreeting()) // "Hello, World!"
}
```

`grpc.WithTransportCredentials(insecure.NewCredentials())` disables TLS — fine for local dev, but use proper TLS in production.

0.9 Run and Test

```
# Start the server
go run ./cmd/server

# In another terminal — test with grpcurl
grpcurl -plaintext localhost:50051 list
# myproject.greeter.v1.GreeterService

grpcurl -plaintext -d '{"name": "World"}' \
  localhost:50051 myproject.greeter.v1.GreeterService/SayHello
# {
#   "greeting": "Hello, World!"
# }
```

0.10 Install Testing Tools

These are optional but highly recommended for development:

```
# grpcurl — like curl, but for gRPC (used in examples above)
go install github.com/fullstorydev/grpcurl/cmd/grpcurl@latest

# evans — interactive REPL for gRPC (tab completion, explore services)
go install github.com/ktr0731/evans@latest
```

Using evans:

```
evans -r repl -p 50051
```

```
# Inside the REPL:
> show service
# GreeterService
> service GreeterService
# Set target service
> call SayHello
# name (TYPE_STRING) => World
# { "greeting": "Hello, World!" }
```

Setup Checklist

Here's everything in one checklist for quick reference:

```
[ ] Go 1.21+ installed
[ ] go mod init (if new project)
[ ] go get google.golang.org/grpc
[ ] go get google.golang.org/protobuf
[ ] buf installed (winget install bufbuild.buf / brew install bufbuild/buf/buf)
[ ] buf.yaml created at project root
[ ] buf.gen.yaml created at project root
[ ] api/proto/ directory created
[ ] At least one .proto file with service + messages
[ ] buf generate runs without errors
[ ] gen/pb/ contains generated .pb.go and _grpc.pb.go files
[ ] Handler struct embeds Unimplemented*Server
[ ] Handler methods implemented
[ ] main.go creates grpc.NewServer, registers handler, calls Serve()
[ ] reflection.Register() called (for dev tooling)
[ ] go build succeeds
[ ] grpcurl or evans can list and call your services
```

---

1. What is gRPC and Why Use It

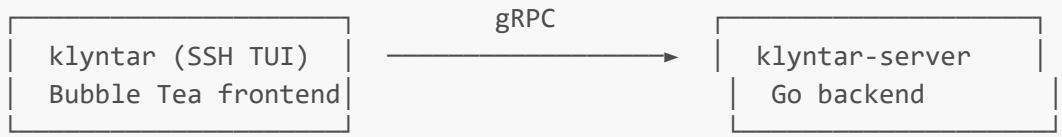
gRPC (gRPC Remote Procedure Calls) is a framework built by Google for making calls between services. Instead of sending JSON over HTTP like REST, gRPC sends **binary data** (Protocol Buffers) over **HTTP/2**.

Think of it this way:

- **REST**: You write an HTTP handler, decide on URL paths, pick HTTP methods, parse JSON manually, and send JSON back.
- **gRPC**: You define your API in a `.proto` file, run a code generator, and implement a Go interface. The framework handles serialization, deserialization, and transport for you.

Why klyntar-server uses gRPC:

This project has two components that talk to each other:



Since both sides are Go programs you control (no browser involved), gRPC is a better fit than REST because:

Advantage	Why it matters here
Type safety	The `.proto` file generates Go structs and interfaces. You can't send the wrong field types — the compiler catches it.
No JSON overhead	Protobuf binary encoding is smaller and faster to parse than JSON.
Streaming	gRPC natively supports streaming (needed for real-time messages later). REST would need WebSockets.
Generated client	The TUI repo gets a ready-made Go client — no need to hand-write HTTP request code.
Contract-first	The `.proto` file *is* the API documentation. Both repos reference it as the source of truth.

2. How gRPC Differs from REST

Here's the same "get user" operation in both styles:

REST (what we had before)

```
Client                                  Server
──────                                  ──────
POST /api/v1/user                       chi router matches the path
Content-Type: application/json          json.NewDecoder parses the body
{"key_fp": "SHA256:abc"}                handler calls controller
                                        controller queries DB
                                        controller returns models.User
                                        json.NewEncoder writes response

◄── 200 OK
    {"id":"...","username":"alice",...}
```

You had to manually:

- Pick the HTTP method and URL path
- Define the JSON shape (and hope client/server agree)
- Parse JSON on both ends
- Map HTTP status codes to error meanings

gRPC (what we have now)

```
Client                                  Server
──────                                  ──────
usersClient.GetUser(ctx,                gRPC framework receives the call
  &GetUserRequest{                      deserializes the protobuf message
    KeyFingerprint: "SHA256:abc",       calls handler.GetUser(ctx, req)
  })                                    handler queries DB
                                        handler returns &User{...}, nil
◄── User{Id:"...", Username:"alice"}    gRPC serializes and sends response
```

The client calls a **Go function** (not an HTTP endpoint). The `.proto` file guarantees both sides agree on the exact shape of `GetUserRequest` and `User`.

---

3. The Toolchain: Protobuf + buf

What is Protocol Buffers (Protobuf)?

Protobuf is a language for defining data structures and services. You write `.proto` files, and a code generator produces Go structs, serialization code, and gRPC interfaces.

What is buf?

`buf` is a modern tool that replaces the older `protoc` compiler. It handles:

- **Linting** your `.proto` files for style issues
- **Breaking change detection** (catches if you accidentally rename a field)
- **Code generation** — runs the protobuf and gRPC Go plugins

How buf is configured in this project

`buf.yaml` — tells buf where the proto files are and what rules to apply:

```yaml
version: v2
modules:
  - path: api/proto          # <-- buf looks here for .proto files
lint:
  use:
    - DEFAULT                # standard linting rules
breaking:
  use:
    - FILE                   # detect breaking changes per-file
```

`buf.gen.yaml` — tells buf what code to generate:

```yaml
version: v2
plugins:
  # Plugin 1: generates Go structs and serialization code
```

```
  - remote: buf.build/protocolbuffers/go
    out: gen/pb                              # output directory
    opt:
      - paths=source_relative               # mirror the proto directory structure

  # Plugin 2: generates gRPC service interfaces and registration code
  - remote: buf.build/grpc/go
    out: gen/pb
    opt:
      - paths=source_relative
```

Two plugins, two kinds of output:

Plugin	What it generates	Output file
`protocolbuffers/go`	Go structs (`User`, `GetUserRequest`, etc.) + marshal/unmarshal	`users.pb.go`
`grpc/go`	Service interface (`UsersServiceServer`) + client + registration	`users_grpc.pb.go`

4. Project Structure After Migration

```
klyntar-server/
├── api/proto/                          # YOU WRITE THESE
│   └── users/v1/
│       └── users.proto                 # API contract definition
│
├── gen/pb/                             # AUTO-GENERATED (do not edit)
│   └── users/v1/
│       ├── users.pb.go                 # Go structs + serialization
│       └── users_grpc.pb.go            # gRPC interfaces + client
│
├── internal/                           # YOU WRITE THESE
│   └── users/
│       └── handler.go                  # implements UsersServiceServer
│
├── server/
│   └── main.go                         # creates gRPC server, registers handlers
│
├── pkg/db/                             # unchanged
│   ├── postgres.go
│   └── transaction.go
├── pkg/cache/                          # unchanged
│   └── redis.go
│
├── buf.yaml                            # buf project config
├── buf.gen.yaml                        # buf code generation config
└── go.mod
```

The workflow is always:

```
.proto file  ──buf generate──►  gen/pb/*.go  ◄──implements──  internal/*/handler.go
  (you write)                   (generated)                    (you write)
```

---

5. Step-by-Step Walkthrough

5.1 The Proto File (`api/proto/users/v1/users.proto`)

This is the most important file. It defines your entire API contract.

```
syntax = "proto3";
package klyntar.users.v1;

option go_package = "example.com/klyntar-server/gen/pb/users/v1";

service UsersService {
  rpc GetUser(GetUserRequest) returns (User);
  rpc CreateUser(CreateUserRequest) returns (CreateUserResponse);
}

message GetUserRequest {
  string key_fingerprint = 1;
}

message CreateUserRequest {
  string username = 1;
  string email = 2;
  string key_fingerprint = 3;
  string device_mac = 4;
}

message CreateUserResponse {
  string id = 1;
}

message User {
  string id = 1;
  string username = 2;
  string email = 3;
  string device_mac = 4;
}
```

Let's break down every line:

```
syntax = "proto3";
```

Use proto3 syntax (the current version). Proto2 is the older syntax — always use proto3 for new projects.

`package klyntar.users.v1;`

A namespacing mechanism. This prevents name collisions if you have a `User` message in multiple proto files. The convention is `<project>.<domain>.<version>`.

`option go_package = "...";`

Tells the Go code generator where to place the generated code and what Go package name to use. This must match the directory structure under `gen/pb/`.

`service UsersService { ... }`

This defines a **gRPC service** — a collection of RPC methods. In the generated Go code, this becomes an **interface** that you must implement:

```
// You'll implement this interface:
type UsersServiceServer interface {
    GetUser(context.Context, *GetUserRequest) (*User, error)
    CreateUser(context.Context, *CreateUserRequest) (*CreateUserResponse, error)
}
```

`rpc GetUser(GetUserRequest) returns (User);`

A single RPC method. It takes one message type as input and returns one message type as output. This is a **unary RPC** — one request, one response (like a normal function call).

Other RPC types you'll use later:

```
// Server streaming — server sends multiple messages back (e.g., feed updates)
rpc GetFeed(GetFeedRequest) returns (stream Post);

// Client streaming — client sends multiple messages (e.g., bulk upload)
rpc UploadPosts(stream Post) returns (UploadResponse);

// Bidirectional streaming — both sides stream (e.g., chat messages)
rpc Chat(stream ChatMessage) returns (stream ChatMessage);
```

`message GetUserRequest { ... }`

A **message** is a data structure — like a Go struct. Each field has:

- A **type** (`string`, `int32`, `bool`, `bytes`, or another message)
- A **name** (`key_fingerprint`)
- A **field number** (`= 1`) — this is the wire format identifier, NOT a default value

Critical rule about field numbers:

> Once you assign a field number and deploy it, NEVER change it or reuse it. Protobuf uses these numbers (not field names) to encode/decode data. Changing them breaks all existing clients.

Common protobuf types:

Protobuf type	Go type	Notes
`string`	`string`	UTF-8
`int32`	`int32`	
`int64`	`int64`	
`bool`	`bool`	
`bytes`	`[]byte`	
`float` / `double`	`float32` / `float64`	
`repeated string`	`[]string`	a list/slice
`map<string, int32>`	`map[string]int32`	
`SomeMessage`	`*SomeMessage`	nested message, becomes a pointer

Why `v1` in the path?

API versioning. If you ever need to make breaking changes to the API, you create `users/v2/users.proto` with new message shapes. The old `v1` clients keep working against `v1` handlers. This is a common pattern in production gRPC services.

5.2 buf Configuration

Already covered in Section 3. The key takeaway:

- `buf.yaml` says "my proto files are in `api/proto/`"
- `buf.gen.yaml` says "generate Go + gRPC code into `gen/pb/`"

5.3 Code Generation

Run this whenever you change a `.proto` file:

```
buf generate
```

This reads all `.proto` files under `api/proto/`, runs both plugins, and writes output to `gen/pb/`. The generated files should be committed to git so the project compiles without needing `buf` installed.

After running, you get:

```
gen/pb/users/v1/
├── users.pb.go          # Go structs: User, GetUserRequest, CreateUserRequest,
etc.
└── users_grpc.pb.go     # Interface: UsersServiceServer, client:
NewUsersServiceClient
```

Never edit files in `gen/` — they get overwritten on every `buf generate`.

5.4 Understanding the Generated Code

`users.pb.go` — Messages (structs)

For each `message` in the proto file, you get a Go struct. For example, `message User` becomes:

```
type User struct {
    Id        string `protobuf:"bytes,1,opt,name=id,proto3" json:"id,omitempty"`
    Username  string `protobuf:"bytes,2,opt,name=username,proto3"
json:"username,omitempty"`
    Email     string `protobuf:"bytes,3,opt,name=email,proto3"
json:"email,omitempty"`
    DeviceMac string `protobuf:"bytes,4,opt,name=device_mac,json=deviceMac,proto3"
json:"device_mac,omitempty"`
}
```

Notice: the proto field `device_mac` (snake_case) becomes `DeviceMac` (PascalCase) in Go. Protobuf always converts to the target language's convention.

Each message also gets **getter methods**:

```
func (x *GetUserRequest) GetKeyFingerprint() string { ... }
```

Always use getters (`req.GetKeyFingerprint()`) instead of direct field access (`req.KeyFingerprint`) — getters are nil-safe (return zero value if the message is nil).

`users_grpc.pb.go` — Service Interface and Client

This file contains the pieces that wire everything together:

1. The server interface — what you must implement:

```
type UsersServiceServer interface {
    GetUser(context.Context, *GetUserRequest) (*User, error)
    CreateUser(context.Context, *CreateUserRequest) (*CreateUserResponse, error)
    mustEmbedUnimplementedUsersServiceServer()
}
```

2. The `Unimplemented` struct — a default implementation that returns "not implemented" errors:

```
type UnimplementedUsersServiceServer struct{}

func (UnimplementedUsersServiceServer) GetUser(context.Context, *GetUserRequest)
(*User, error) {
    return nil, status.Error(codes.Unimplemented, "method GetUser not
implemented")
}
```

You embed this in your handler (explained in the next section).

3. The registration function — used in `main.go` to hook your handler into the gRPC server:

```
func RegisterUsersServiceServer(s grpc.ServiceRegistrar, srv UsersServiceServer)
```

4. The client — used by the klyntar TUI to call this server:

```
type UsersServiceClient interface {
    GetUser(ctx context.Context, in *GetUserRequest, opts ...grpc.CallOption)
(*User, error)
    CreateUser(ctx context.Context, in *CreateUserRequest, opts
...grpc.CallOption) (*CreateUserResponse, error)
}

func NewUsersServiceClient(cc grpc.ClientConnInterface) UsersServiceClient
```

The klyntar TUI repo will use this client like:

```
conn, _ := grpc.Dial("localhost:50051", grpc.WithInsecure())
client := userspb.NewUsersServiceClient(conn)

user, err := client.GetUser(ctx, &userspb.GetUserRequest{
    KeyFingerprint: "SHA256:abc123",
})
```

5.5 Implementing the Handler (`internal/users/handler.go`)

This is where your business logic lives. Let's walk through the entire file:

```
package users

import (
    "context"
    "database/sql"

    userspb "example.com/klyntar-server/gen/pb/users/v1"
    "example.com/klyntar-server/pkg/db"
    "google.golang.org/grpc/codes"
    "google.golang.org/grpc/status"
)
```

`userspb` **import alias**: The generated package is `v1`, which is generic. Aliasing it to `userspb` makes the code readable — `userspb.User` is clearer than `v1.User`.

```
type Handler struct {
    userspb.UnimplementedUsersServiceServer
    db *sql.DB
}
```

Why embed `UnimplementedUsersServiceServer`?

This is a **forward compatibility** pattern. Imagine you add a new RPC method to the proto file later:

```
service UsersService {
    rpc GetUser(...) returns (...);
    rpc CreateUser(...) returns (...);
    rpc DeleteUser(...) returns (...);  // NEW
}
```

After regenerating, the `UsersServiceServer` interface now has three methods. Without the embed, your code **wouldn't compile** until you implement `DeleteUser`. With the embed, the `Unimplemented` struct provides a default `DeleteUser` that returns a "not implemented" error — your code still compiles, and you can implement the new method at your own pace.

Rule: always embed by value (not pointer). This is why it's `userspb.UnimplementedUsersServiceServer` and not `*userspb.UnimplementedUsersServiceServer`.

```
func NewHandler(database *sql.DB) *Handler {
    return &Handler{db: database}
}
```

Constructor that injects the database connection. This is the same dependency injection pattern used in the old REST controllers, just cleaner.

GetUser implementation

```
func (h *Handler) GetUser(ctx context.Context, req *userspb.GetUserRequest)
(*userspb.User, error) {
    // 1. Validate input
    if req.GetKeyFingerprint() == "" {
        return nil, status.Error(codes.InvalidArgument, "key_fingerprint is
required")
    }

    // 2. Query database
    var user userspb.User
    err := db.WithTx(h.db, ctx, func(tx *sql.Tx) error {
        return tx.QueryRowContext(ctx,
            "SELECT id, username, email, device_mac FROM users WHERE
key_fingerprint = $1",
            req.GetKeyFingerprint(),
        ).Scan(&user.Id, &user.Username, &user.Email, &user.DeviceMac)
    })

    // 3. Handle errors with gRPC status codes
    if err == sql.ErrNoRows {
        return nil, status.Error(codes.NotFound, "user not found")
    }
    if err != nil {
        return nil, status.Error(codes.Internal, "failed to get user")
    }

    // 4. Return the protobuf message directly
    return &user, nil
}
```

Key differences from the old REST handler:

REST version	gRPC version
Parse JSON body manually with `json.NewDecoder`	Request is already parsed — `req` is a typed Go struct
Set HTTP status with `w.WriteHeader(404)`	Return `status.Error(codes.NotFound, ...)`
Write JSON response with `json.NewEncoder`	Return the protobuf struct — framework serializes it
Handler signature: `func(w http.ResponseWriter, r *http.Request)`	Handler signature: `func(ctx context.Context, req *Message) (*Response, error)`

Notice we scan directly into `userspb.User` fields. The generated struct fields (`Id`, `Username`, `Email`, `DeviceMac`) are regular Go strings — you can use them like any struct.

CreateUser implementation

```
func (h *Handler) CreateUser(ctx context.Context, req *userspb.CreateUserRequest)
(*userspb.CreateUserResponse, error) {
    if req.GetUsername() == "" || req.GetEmail() == "" || req.GetKeyFingerprint()
== "" {
        return nil, status.Error(codes.InvalidArgument, "username, email, and
key_fingerprint are required")
    }

    var id string
    err := db.WithTx(h.db, ctx, func(tx *sql.Tx) error {
        return tx.QueryRowContext(ctx,
            "INSERT INTO users (username, email, key_fingerprint, device_mac)
VALUES ($1, $2, $3, $4) RETURNING id",
            req.GetUsername(), req.GetEmail(), req.GetKeyFingerprint(),
req.GetDeviceMac(),
        ).Scan(&id)
    })
    if err != nil {
        return nil, status.Error(codes.Internal, "failed to create user")
    }

    return &userspb.CreateUserResponse{Id: id}, nil
}
```

Same pattern: validate, query, return typed response. The `RETURNING id` clause lets us get the auto-generated UUID back from Postgres and return it in the response.

5.6 Wiring the gRPC Server (`server/main.go`)

The infrastructure setup (Postgres, Redis, migrations, env loading) is identical. What changed is the server setup at the bottom:

Before (REST):

```
application := &app.Application{Config: cfg, DbConnector: sqlDB}
application.RegisterRoutes(routes.RegisterUserRoutes)
mux := application.Mount()              // chi router
application.Run(mux)                     // http.ListenAndServe
```

After (gRPC):

```
// 1. Create a TCP listener
lis, err := net.Listen("tcp", fmt.Sprintf(":%d", cfg.GRPCPort))

// 2. Create a gRPC server
grpcServer := grpc.NewServer()
```

```
// 3. Create your handler and register it
usersHandler := users.NewHandler(sqlDB)
userspb.RegisterUsersServiceServer(grpcServer, usersHandler)

// 4. Enable reflection (for development tools)
reflection.Register(grpcServer)

// 5. Start serving
grpcServer.Serve(lis)
```

Why `net.Listen` + `grpcServer.Serve`?

In REST, `http.ListenAndServe(":3000", handler)` does both — creates a listener and serves. gRPC separates these steps because gRPC runs over HTTP/2, and you might want to configure the listener separately (e.g., TLS, Unix sockets).

What is `reflection.Register`?

gRPC reflection lets tools like `grpcurl` and `evans` discover your services at runtime — like a self-describing API. Without it, you'd need to pass the `.proto` files to these tools manually. **Enable it in development, disable it in production** (it exposes your API schema).

Graceful shutdown:

```
go func() {
    sigCh := make(chan os.Signal, 1)
    signal.Notify(sigCh, syscall.SIGINT, syscall.SIGTERM)
    <-sigCh                          // block until Ctrl+C or kill signal
    slog.Info("shutting down gRPC server")
    grpcServer.GracefulStop()        // finish in-flight RPCs, then stop
}()
```

This runs in a goroutine so it doesn't block `Serve()`. When you press Ctrl+C:

1. The signal arrives on `sigCh`
2. `GracefulStop()` tells the server to stop accepting new RPCs
3. It waits for all active RPCs to complete
4. `Serve()` returns, and `main()` exits

---

6. What Got Removed and Why

Deleted	Why
`app/app.go`	The chi router, HTTP middleware, and `http.Server` setup. gRPC has its own server and middleware system (interceptors).

Deleted	Why
`api/routes/users.go`	HTTP route handlers that parsed JSON and called controllers. Replaced by `internal/users/handler.go`.
`api/routes/health.go`	HTTP health check. gRPC has its own health checking protocol (you'll add this later with `grpc_health_v1`).
`api/controllers/crudUser.go`	Database logic that was coupled to `http.Request` and `app.Application`. The same SQL now lives in the handler, taking `context.Context` directly.
`models/user.go`	The `User` struct. Replaced by the generated `userspb.User` from the proto file.
`utils/jsonWriter.go`	JSON response helper. gRPC doesn't use JSON — protobuf handles serialization.
`chi` dependency in go.mod	No longer needed.

7. gRPC Error Handling

REST uses HTTP status codes (200, 404, 500). gRPC has its own **status codes** defined in `google.golang.org/grpc/codes`:

```
import (
    "google.golang.org/grpc/codes"
    "google.golang.org/grpc/status"
)

// Return an error with a gRPC status code:
return nil, status.Error(codes.NotFound, "user not found")
```

Common codes and their REST equivalents:

gRPC Code	HTTP Equivalent	When to use
`codes.OK`	200	Success (returned automatically when err is nil)
`codes.InvalidArgument`	400	Bad input from client
`codes.NotFound`	404	Resource doesn't exist
`codes.AlreadyExists`	409	Duplicate (e.g., username taken)
`codes.PermissionDenied`	403	Not allowed
`codes.Unauthenticated`	401	Not logged in
`codes.Internal`	500	Server bug
`codes.Unavailable`	503	Temporary failure (client should retry)

gRPC Code	HTTP Equivalent	When to use
`codes.Unimplemented`	501	RPC method not implemented yet

On the client side, you check errors like this:

```
user, err := client.GetUser(ctx, req)
if err != nil {
    st, ok := status.FromError(err)
    if ok {
        switch st.Code() {
        case codes.NotFound:
            // user doesn't exist
        case codes.InvalidArgument:
            // bad request
        default:
            // something else
        }
    }
}
```

---

8. Testing Your gRPC Server

Using grpcurl (CLI tool, like curl for gRPC)

```
# Install
go install github.com/fullstorydev/grpcurl/cmd/grpcurl@latest

# List all services (requires reflection to be enabled)
grpcurl -plaintext localhost:50051 list

# List methods in a service
grpcurl -plaintext localhost:50051 list klyntar.users.v1.UsersService

# Call CreateUser
grpcurl -plaintext -d '{
  "username": "alice",
  "email": "alice@example.com",
  "key_fingerprint": "SHA256:abc123",
  "device_mac": "AA:BB:CC:DD:EE:FF"
}' localhost:50051 klyntar.users.v1.UsersService/CreateUser

# Call GetUser
grpcurl -plaintext -d '{
  "key_fingerprint": "SHA256:abc123"
}' localhost:50051 klyntar.users.v1.UsersService/GetUser
```

Using evans (interactive REPL)

```
# Install
go install github.com/ktr0731/evans@latest

# Connect in REPL mode
evans -r repl -p 50051

# Inside the REPL:
> package klyntar.users.v1
> service UsersService
> call GetUser
key_fingerprint (TYPE_STRING) => SHA256:abc123
```

Writing Go tests

```
func TestGetUser(t *testing.T) {
    // Set up a test gRPC server
    lis := bufconn.Listen(1024 * 1024)
    srv := grpc.NewServer()
    userspb.RegisterUsersServiceServer(srv, users.NewHandler(testDB))
    go srv.Serve(lis)

    // Create a client that connects to the test server
    conn, _ := grpc.DialContext(ctx, "bufnet",
        grpc.WithContextDialer(func(ctx context.Context, s string) (net.Conn,
error) {
            return lis.Dial()
        }),
        grpc.WithTransportCredentials(insecure.NewCredentials()),
    )
    client := userspb.NewUsersServiceClient(conn)

    // Call the RPC
    user, err := client.GetUser(ctx, &userspb.GetUserRequest{
        KeyFingerprint: "SHA256:abc123",
    })

    // Assert
    assert.NoError(t, err)
    assert.Equal(t, "alice", user.Username)
}
```

The `bufconn` package lets you test gRPC without opening a real network port — everything happens in memory.

---

9. How to Add a New Service

When you're ready to add the Voids (communities) service, follow this recipe:

Step 1: Create the proto file

```
mkdir -p api/proto/voids/v1
```

Write `api/proto/voids/v1/voids.proto`:

```
syntax = "proto3";
package klyntar.voids.v1;

option go_package = "example.com/klyntar-server/gen/pb/voids/v1";

service VoidsService {
  rpc CreateVoid(CreateVoidRequest) returns (Void);
  rpc GetVoid(GetVoidRequest) returns (Void);
  rpc ListVoids(ListVoidsRequest) returns (ListVoidsResponse);
}

message Void {
  string id = 1;
  string name = 2;
  string description = 3;
  string creator_id = 4;
}

message CreateVoidRequest {
  string name = 1;
  string description = 2;
  string creator_id = 3;
}

message GetVoidRequest {
  string id = 1;
}

message ListVoidsRequest {
  int32 limit = 1;
  int32 offset = 2;
}

message ListVoidsResponse {
  repeated Void voids = 1;
}
```

Step 2: Generate code

```
buf generate
```

New files appear at `gen/pb/voids/v1/`.

Step 3: Implement the handler

Create `internal/voids/handler.go`:

```
package voids

import (
    "context"
    "database/sql"

    voidspb "example.com/klyntar-server/gen/pb/voids/v1"
    // ...
)

type Handler struct {
    voidspb.UnimplementedVoidsServiceServer
    db *sql.DB
}

func NewHandler(database *sql.DB) *Handler {
    return &Handler{db: database}
}

func (h *Handler) CreateVoid(ctx context.Context, req *voidspb.CreateVoidRequest)
(*voidspb.Void, error) {
    // your implementation here
}

// ... implement other methods
```

Step 4: Register in main.go

```
import voidspb "example.com/klyntar-server/gen/pb/voids/v1"
import "example.com/klyntar-server/internal/voids"

// In main(), after creating grpcServer:
voidsHandler := voids.NewHandler(sqlDB)
voidspb.RegisterVoidsServiceServer(grpcServer, voidsHandler)
```

That's it. Four steps, every time.

---

10. Key Concepts Reference

Glossary

Term	Meaning
Protobuf	Protocol Buffers — a binary serialization format and schema language
Proto file	A `.proto` file defining messages and services
Message	A data structure in protobuf (becomes a Go struct)
Service	A collection of RPC methods (becomes a Go interface)
RPC	Remote Procedure Call — a method you can call across the network
Unary RPC	One request, one response (what we have now)
Streaming RPC	One or both sides send multiple messages (used for feeds, chat)
buf	Tool that lints, validates, and generates code from proto files
Reflection	Lets tools discover your gRPC API at runtime without needing proto files
Interceptor	gRPC's version of middleware (logging, auth, etc.) — you'll add these later
Status code	gRPC's error codes (like HTTP status codes but different set)
Channel	A gRPC client connection (the `grpc.ClientConn` object)

Files you edit vs. files you don't

File	Edit?	Notes
`api/proto/**/*.proto`	Yes	This is your API definition
`buf.yaml` / `buf.gen.yaml`	Rarely	Only when adding new plugins or changing output paths
`gen/pb/**/*.go`	**Never**	Regenerated by `buf generate`
`internal/*/handler.go`	Yes	Your business logic
`server/main.go`	Yes	When registering new services

Useful links

- Protocol Buffers Language Guide (proto3) — official reference for `.proto` file syntax
- gRPC Go Quick Start — official getting started guide
- gRPC Go API Reference — Go package documentation
- gRPC Status Codes — full list of status codes and when to use them
- buf Documentation — buf tool reference
- grpcurl — command-line gRPC client
- evans — interactive gRPC REPL